

CodeArena

Projektausschreibung

Prof. Dr. rer. nat. Jörg Striegnitz

Januar 9, 2026

CodeArena

Prof. Dr. rer. nat. Jörg Striegnitz

Fachbereich Medizintechnik und Technomathematik – Fachhochschule Aachen

Wintersemester 2025/26

Kreditierung: 15 ECTS (ca. 375–450 Stunden Workload)

Projekt-Info: „CodeArena” – Progressive Web-App zur sicheren Ausführung von Code-Snippets in mehreren Programmiersprachen mit Sharing-Funktion

1 Hintergrund & Motivation

Das schnelle Ausprobieren von Code-Snippets ist ein alltäglicher Bestandteil der Softwareentwicklung und des Informatikstudiums. Tools wie Godbolt (Compiler Explorer), Replit oder Go Playground ermöglichen die Ausführung von Code im Browser, ohne lokale Entwicklungsumgebungen einrichten zu müssen.

Eine eigene Code-Sandbox bietet mehrere Vorteile: vollständige Kontrolle über die Ausführungsumgebung, Erweiterbarkeit um eigene Sprachen, Integration in Lehrmaterialien und die Möglichkeit, sichere Container-Isolation in der Praxis zu erlernen. Das Projekt kombiniert Frontend-Entwicklung, Container-Orchestrierung mit Go und grundlegende Sicherheitsanforderungen.

2 Zielsetzung & Lernziele

2.1 Zielsetzung

Entwicklung einer Progressive Web App (PWA) zur sicheren Ausführung von Code-Snippets in mindestens drei Programmiersprachen (Go, Python, C++). Die Anwendung soll Code in isolierten Containern ausführen, Ergebnisse anzeigen und das Teilen von Snippets ermöglichen.

2.2 3.2 Lernziele

Kompetenzbereich	Erwarteter Lernerfolg
Frontend-Entwicklung	TypeScript-basiertes React-Projekt (Vite), Monaco-Editor-Integration, Service-Worker.
Backend (Go)	Entwicklung eines Orchestrators für Container- basierte Code-Ausführung, REST-API mit Go Standard Library (net/http).

Kompetenzbereich	Erwarteter Lernerfolg
Container-Sicherheit	Sandbox-Isolation mit Docker, Ressourcenlimits (CPU, Memory, Zeit), Netzwerk-Isolation.
Server-Administration	Einrichtung eines Ubuntu-Servers, Docker-Engine, Docker-Compose, Reverse-Proxy (Traefik/Nginx), Firewall.
Infrastructure as Code	Schreiben von Dockerfiles für Compiler-Images, docker-compose.yml, GitHub-Action für Deploy.
Datenbank	PostgreSQL oder SQLite für Snippet-Speicherung und Sharing-Links.
Qualitätssicherung	Unit-Tests (Go testing, Vitest), End-to-End-Tests (Playwright), CI-Pipeline.
Dokumentation	Erstellung eines technischen Handbuchs, Installationsanleitung und einer 10-minütigen Demo-Präsentation.

3 Leistungsbeschreibung

3.1 Funktionale Anforderungen – Pflichtteil

ID	Anforderung
F1	Code-Editor – Monaco-Editor mit Syntax-Highlighting für alle unterstützten Sprachen, Zeilennummern.
F2	Sprachauswahl – Mindestens 2 Sprachen: Go, Python (C++ optional).
F3	Code-Ausführung – Kompilierung und Ausführung in isolierten Containern, Anzeige von stdout/stderr, Exit-Code.
F4	Ressourcenlimits – Timeout (max. 10s), Memory-Limit (max. 256 MB), Netzwerk-Isolation.
F5	Eingabe-Unterstützung – Möglichkeit, stdin-Eingaben für das Programm zu definieren.
F6	Sharing – Generierung von Kurzlinks zum Teilen von Snippets.
F7	Snippet-Bibliothek – Sammlung von Beispiel-Snippets pro Sprache (Hello World, Sortierung).
F8	Responsive Design – Nutzbar auf Desktop und Tablet.

3.2 Funktionale Anforderungen – Optional (Bonus)

ID	Anforderung
O1	Weitere Sprache – Rust oder JavaScript als vierte Sprache.
O2	Vergleichsmodus – Derselbe Algorithmus in mehreren Sprachen nebeneinander.
O3	Ausführungshistorie – Anzeige der letzten Ausführungen (Session-basiert).
O4	Compiler-Optionen – Auswahl von C++-Standards (C++17, C++20), Optimierungsstufen.

3.3 Unterstützte Sprachen

Sprache	Compiler/Runtime	Pflicht/Optional
Go	go 1.21+	Pflicht
Python	python 3.11+	Pflicht
C++	g++ 13+ oder clang++	Pflicht
Rust	rustc	Optional
JavaScript	Node.js	Optional

Sprache	Compiler/Runtime	Pflicht/Optional
---------	------------------	------------------

3.4 Nicht-funktionale Anforderungen

ID	Anforderung
NF1	PWA – Service-Worker, Manifest, Installierbarkeit (Add-to-Home-Screen).
NF2	TypeScript – Durchgängige Typisierung im Frontend; ESLint + Prettier.
NF3	Go-Backend – Orchestrator für Container-Ausführung, REST-API, synchrone Verarbeitung.
NF4	Container-Isolation – Jede Ausführung in eigenem Container, keine persistenten Dateisysteme, Netzwerk deaktiviert.
NF5	Sicherheit – Ressourcenlimits strikt enforced, kein Root im Container, Firewall (ufw).
NF6	Performance – Kompilierung + Ausführung eines Hello-World < 5s.
NF7	Reverse-Proxy – Traefik (empfohlen) oder Nginx als Gateway; lauffähiges Setup via Docker-Compose; TLS (automatisch via Traefik).
NF8	Testing – Go-Tests (Unit), Vitest (Frontend), mindestens ein E2E-Test (Playwright).
NF9	Dokumentation – README, Installations-/Betriebsanleitung, Sicherheitskonzept, Entwickler-Handbuch (max. 5 Seiten).

4 Technische Rahmenbedingungen

Komponente	Vorgabe
Frontend	React + Vite, TypeScript ≥ 5.0, Monaco Editor. State: Context API oder 'Zustand'.
Code-Editor	Monaco Editor mit Language-Support für alle Zielsprachen.
Backend	Go ≥ 1.21 (Go Standard Library), Container- Orchestrierung via Docker SDK für Go.
Datenbank	PostgreSQL oder SQLite für Snippets und Sharing-Links.
Container-Runtime	Docker oder Podman mit Standard-Isolation.
Compiler-Images	Separate Docker-Images pro Sprache (golang, python, gcc).
Server-OS	Ubuntu 22.04 LTS (oder Debian-basiert).
Container-Orchestrierung	Docker-Engine oder Podman.
Build-Automation	Makefile oder Taskfile (https://taskfile.dev).
Reverse-Proxy	Traefik (empfohlen) oder Nginx; automatisches TLS via Let's Encrypt.
CI/CD	GitHub Actions → Build Docker-Images → SSH-Deploy auf VPS.
Testing	Go testing (Unit), Vitest (Frontend), Playwright (E2E).
Lizenz	Gesamter Quellcode unter Apache 2.0 oder MIT.

5 Arbeitspakete & Meilensteine

WP	Inhalt	Wochen	Ergebnis
WP-1	Anforderungsanalyse, Pflichtenheft, Sicherheitskonzept	1–2	Pflichtenheft, Sicherheitsdoku
WP-2	Systemarchitektur, Container- Strategie, API-Design	3	Architektur- Dokument

WP	Inhalt	Wochen	Ergebnis
WP-3	Frontend-Skeleton: React + Monaco Editor, Sprachauswahl	4–5	Git-Repo mit funktionierendem Editor
WP-4	Compiler-Images: Dockerfiles für Go, Python, C++	6	Getestete Docker- Images
WP-5	Orchestrator (Go): Container-Spawn, Timeout, Resource-Limits	7–8	REST-Endpoint /api/execute, Tests
WP-6	Sharing & Persistenz: PostgreSQL, Kurzlinks, Snippet-Bibliothek	9–10	Sharing-Funktion, Beispiel-Snippets
WP-7	Docker-Compose + Reverse-Proxy Setup, PWA-Finalisierung	11	Lauffähiges lokal-isiertes System
WP-8	Tests, Dokumentation, Abschlusspräsentation	12–13	Test-Reports, Handbuch, Präsentation

Der Gesamtzeitraum beträgt ca. **13 Wochen** Kernentwicklung. Pufferzeit für unvorhergesehene Probleme ist eingeplant.

6 Bewertungskriterien

Kriterium	Gew.	Bemerkungen
Konzept & Pflichtenheft	12%	Vollständigkeit, Sicherheits- konzept, Risiko-Management.
Architektur & Infrastruktur	15%	Container-Strategie, Docker- Compose, Reverse-Proxy, TLS.
Implementierung Frontend	18%	Editor-Integration, UX, Responsivität, PWA-Erfüllung.
Implementierung Backend	22%	Orchestrator, Container-Isolation, API-Qualität.
Sicherheit	8%	Container-Isolation, Ressourcen- limits.
Testing & CI/CD	10%	Unit- und E2E-Tests, automatisierter Deploy auf VPS.
Dokumentation	10%	Installations-/Betriebsanleitung, Sicherheitsdoku.
Präsentation & Demo	5%	Live-Demo über HTTPS-URL, klare Erläuterung der Architektur.
Bonus (Optional)	bis 5%	Weitere Sprachen (C++, Rust), Vergleichsmodus, Compiler-Optionen, Deployment auf VPS via HTTPS.

7 Abgabeformate

Artefakt	Format / Details
Quellcode	Git-Repository (GitHub/GitLab) – Branch main enthält finalen Stand, Tag v1.0.
Docker-Compose & Dockerfiles	docker-compose.yml, Orchestrator- Dockerfile, Compiler-Dockerfiles; alle im Repo.

Artefakt	Format / Details
Proxy-Konfiguration	traefik.yml / Labels in docker-compose oder nginx.conf mit Rate-Limiting.
Deploy-Skript / CI-Workflow	GitHub-Action-Datei (.github/workflows/deploy.yml).
Sicherheitsdokumentation	Beschreibung der Isolation-Maßnahmen (1-2 Seiten).
Dokumentation	README, Installations-Guide, Entwickler- Handbuch (max. 5 Seiten).
Test-Reports	Go-Test-Coverage, Vitest-Coverage, Playwright-Log.
Präsentation & Demo-Video	PowerPoint/Keynote (max. 10 Folien) + kurzes Video (5-7 min).

Alle Artefakte werden im GitLab der FH-Aachen zugänglich gemacht.

8 Hinweis zu Datenschutz & Sicherheit

1. **HTTPS** – Der Server ist zwingend über TLS erreichbar (Let's Encrypt).
2. **Container-Isolation** – Jede Code-Ausführung erfolgt in einem isolierten Container ohne Netzwerkzugriff.
3. **Ressourcenlimits** – Strikte Limits für CPU (1 Core), Memory (256 MB), Zeit (10s) verhindern Denial-of-Service.
4. **Kein persistenter Speicher** – Container haben keinen Zugriff auf das Host-Dateisystem.
5. **Rate-Limiting** – API-Endpunkte sind gegen übermäßige Anfragen geschützt.
6. **Firewall** – Auf dem Ubuntu-Server wird ufw nur für die Ports 80, 443 und 22 geöffnet.
7. **Code-Inhalte** – Geteilte Snippets sind öffentlich; Nutzer werden darauf hingewiesen, keine sensiblen Daten einzugeben.

9 Fortsetzung als Bachelorarbeit

Das Projekt bietet vielfältige Erweiterungsmöglichkeiten für eine anschließende Bachelorarbeit (12 ECTS):

9.1 Mögliche Themen

Thema	Beschreibung
gVisor/seccomp-Hardening	Zusätzliche Kernel-Isolation mit gVisor- Runtime oder maßgeschneiderten seccomp- Profilen, Analyse von Container-Escape- Vektoren.
WebAssembly Sandbox	Ausführung von Code-Snippets in einer ultraleichten WASM-Runtime (z.B. Wazero) als performante Alternative zu Docker-Containern.
Security-Audit & eBPF	Systematische Sicherheitsanalyse der Sandbox, Monitoring von Syscalls via eBPF zur Erkennung von Ausbruchsversuchen.
Job-Queue-Architektur	Skalierbare Ausführung mit Redis/NATS, Worker-Pool, horizontale Skalierung, Load-Balancing.
Vergleichsmodus & Benchmarking	Multi-Language-View, automatische Performance-Vergleiche, Komplexitätsanalyse.
Embedded-Modus	Integration in Vorlesungsmaterialien via iframe, LTI-Schnittstelle für Moodle/ILIAS.

Thema	Beschreibung
WebAssembly-Ausführung	Alternative Sandbox via WASM: Sprachen zu WASM kompilieren, Browser-basierte Ausführung ohne Server.
Nutzer-Accounts	Authentifizierung, persönliche Snippet- Sammlungen, private Snippets.

9.2 Wissenschaftlicher Fokus

Eine Bachelorarbeit könnte den Fokus auf **Container-Sicherheit** legen: Systematische Analyse von Escape-Vektoren, Vergleich verschiedener Isolation-Techniken (Docker, gVisor, Firecracker), Penetration-Testing der eigenen Implementierung.

10 Ressourcen & Literatur

11 Thema

React + TypeScript (Vite)
 Progressive Web Apps
 Monaco Editor
 Go Standard Library
 Docker SDK für Go
 Docker-Sicherheit
 seccomp-Profil
 Docker & Docker-Compose
 Reverse-Proxy
 Nginx (Alternative)
 Godbolt (Inspiration)
 Go Playground

12 Quelle

<https://vitejs.dev/guide/>
<https://web.dev/progressive-web-apps/>
<https://microsoft.github.io/monaco-editor/>
<https://pkg.go.dev/net/http>
<https://pkg.go.dev/github.com/docker/docker/client>
<https://docs.docker.com/engine/security/>
<https://docs.docker.com/engine/security/seccomp/>
<https://docs.docker.com/get-started/>
<https://traefik.io/traefik/>
<https://nginx.org/>
<https://godbolt.org/>
<https://go.dev/play/>

13 Ansprechpartner

Name	E-Mail
Prof. Dr. Jörg Striegnitz	striegnitz@fh-aachen.de